# CLASS ASSIGNMENT 5
# ID : 12341550

## PART 1

### STEP 1

**FILE :** proc.h

```
int page_faults;
```

### STEP 2

**FILE :** proc.c

*in allocproc*

```
p->page_faults = 0;
```

*in growproc*

```
sz += n;
```

### STEP 3

**FILE :** syscall.h

```
#define SYS_getpagefaults 31;
```

**FILE :** sysproc.c

```
int sys_getpagefaults(void) {
  struct proc *p = myproc();
  return p->page_faults;
}
```

**FILE :** syscall.c

```
extern int sys_getpagefaults(void);
…
[SYS_getpagefaults] sys_getpagefaults,
```

**FILE :** user.h

```
int getpagefaults(void);
```

**FILE :** usys.S

```
SYSCALL(getpagefaults)
```

## PART 2

**FILE :** vm.c

```
int vmfault(pde_t *pgdir, uint va, int write)
{
  struct proc *p = myproc();
  char *mem;
  if (va >= p->sz)
    return -1;
  va = PGROUNDDOWN(va);
  if (walkpgdir(pgdir, (void *)va, 0))
    return 0;
  mem = kalloc();
  if (mem == 0)
    return -1;
  memset(mem, 0, PGSIZE);
  if (mappages(pgdir, (void *)va, PGSIZE, V2P(mem),
PTE_W | PTE_U) < 0)
  {
    kfree(mem);
```

```
    return -1;
  }
  return 0;
}
```

**FILE :** trap.c

*in trap() , added new switch case*

```
case T_PGFLT: {
  struct proc *p = myproc();
  if (p && (tf->cs & 3) == DPL_USER) {
    p->page_faults++;

    if (vmfault(p->pgdir, rcr2(), tf->err & 2) < 0) {
      p->killed = 1;
    }
    switchuvm(p);
    return;
  }
  break;
}
```

**FILE :** defs.h

```
int           vmfault(pde_t *pagetable, uint va, int write);
```

## PART 3

**FILE :** tlbrun.c (new)

```
#include "types.h"
#include "user.h"
#define PAGESIZE 4096
#define MAXPAGES 1024
```

```
int main()
{
   int jump = PAGESIZE / sizeof(int);
   printf(1, "PageCount\tTrials\tTicks\tPageFaults\n");
   for (int numpages = 1; numpages <= MAXPAGES;
numpages *= 2)
   {
      int trials = 5000000;
      int faults_before = getpagefaults();
      int start = uptime();
      int *arr = (int *)sbrk(numpages * PAGESIZE);
      if (arr == (void *)-1)
         exit();
      for (int t = 0; t < trials; t++)
      {
         for (int i = 0; i < (numpages / 2) * jump; i += jump)
         {
            // #WRITE YOUR CODE HERE: Access the page
to trigger faults
            arr[i] += 1;
         }
      }
      int end = uptime();
      int faults_after = getpagefaults();
      printf(1, "%d\t %d\t %d\t %d\n", numpages, trials, end -
start, faults_after - faults_before);
   }
   exit();
}
```

**FILE :** tlbtest.c

```
#include "types.h"
#include "user.h"
```

```
#define PAGESIZE 4096
#define MAXPAGES 1024

int main(int argc, char *argv[]) {
    if (argc < 3) {
        printf(1, "Usage: tlbtest <numpages> <trials>\n");
        exit();
    }

    int numpages = atoi(argv[1]);
    int trials = atoi(argv[2]);

    if (numpages < 1 || numpages > MAXPAGES) {
        printf(1, "numpages out of range (1..%d)\n",
MAXPAGES);
        exit();
    }

    int jump = PAGESIZE / sizeof(int);
    int i, t;

    int *arr = (int *)sbrk(numpages * PAGESIZE);
    if (arr == (void *)-1) {
        printf(1, "sbrk failed for %d pages\n", numpages);
        exit();
}

    int pf_before = getpagefaults();

    uint start_ticks = uptime();

    for (t = 0; t < trials; t++) {
        for (i = 0; i < (numpages / 2) * jump; i += jump) {
            arr[i] += 1; // Access page to trigger page fault on
first touch
```

```
    }
  }

  uint end_ticks = uptime();

  int pf_after = getpagefaults();

  printf(1, "Pages: %d\tTrials: %d\tTicks: %d\tPageFaults: %d\n",
         numpages, trials, end_ticks - start_ticks, pf_after - pf_before);

  exit();
}
```

# OUTPUT :

